

Qwen3-Coder-Next Tier 3

Part 9 Detailed Implementation Specification

Repository Intelligence

Internal engineering specification

Defines the implementation of the repository understanding layer that feeds later graph, compression, and coordination components.

Document intent
This document tells an engineer exactly how to build the Repository Intelligence layer. It is not a roadmap and not an architecture overview. It is the implementation specification that bridges the finalized architecture and later deep-dive system parts.

1. Purpose

This part exists to create a durable, queryable understanding of a repository before the system attempts deeper semantic reasoning.

Its main job is to answer practical repository questions quickly: What files exist? How is the project organized? Which files are likely relevant to a task? What changed since the last snapshot?

The output of this part becomes the foundation for Part 10, the Code Knowledge Graph, and for later context compression, routing, and evaluation features.

Why this matters
Without repository intelligence, downstream agents burn context reading irrelevant files, misidentify ownership boundaries, and produce brittle plans. With it, they can start from structured repository facts instead of raw file dumps.

2. Scope

Included: repository discovery, file inventory, path normalization, file classification, lightweight dependency hints, file and folder summaries, incremental re-indexing, persistent snapshots, and a query interface for other agents.

Excluded: full AST graph construction, execution tracing, semantic call-path reasoning, test synthesis, deployment orchestration, and all Code Knowledge Graph responsibilities.

This part may read source files and metadata, but it must stay intentionally shallow. Its output should be a strong repository map, not a full graph engine.

3. System Overview

The runtime starts with a repository root and produces a structured repository model. That model is stored, refreshed, and queried by later agents.

The flow below shows the intended high-level path.



The repository intelligence layer is read-heavy and update-aware. Most operations should be deterministic and repeatable, with minimal hidden state.

4. Internal Architecture

The layer is organized around a small set of responsibilities. Each component should do one thing and expose one narrow interface.

The implementation should prefer incremental updates over full rescans whenever repository state can be derived from git metadata or file hashes.

Component	Responsibility
RepoScanner	Walks the repository tree, applies exclusion rules, and produces the raw file set.
PathNormalizer	Converts repo-relative and OS-specific paths into canonical identifiers.
FileClassifier	Labels files and folders by type, language, role, and stability.
DependencyHintExtractor	Collects shallow import and include hints without building a full graph.
SummaryGenerator	Creates concise folder and file summaries for downstream agents.
ManifestStore	Persists snapshots, hashes, summaries, and version metadata.
IncrementalIndexer	Detects what changed and updates only affected records.
RepositoryQueryService	Serves structured queries from the stored manifest.

Component pipeline
RepoScanner -> PathNormalizer -> FileClassifier -> DependencyHintExtractor -> SummaryGenerator -> ManifestStore -> RepositoryQueryService

Required abstractions include a stable repository identifier, a file record model, a folder record model, a dependency hint model, and a snapshot record.

The interfaces must be designed for future consumers. Part 10 should be able to reuse the stored repository data instead of re-scanning the tree.

5. Project Structure

A clean directory structure keeps repository intelligence separate from graph logic and from the rest of the runtime.

Folder	Purpose
repo_intelligence/	Top-level package for repository discovery, classification, summaries, and persistence.
repo_intelligence/scanners/	Tree walking, ignore rules, and filesystem discovery logic.
repo_intelligence/classification/	Language, file-type, and role classification logic.
repo_intelligence/dependencies/	Shallow dependency extraction and hint normalization.
repo_intelligence/summaries/	Folder and file summary generation.
repo_intelligence/storage/	Snapshot storage, manifest serialization, and versioning.
repo_intelligence/api/	Query interfaces used by planner, research, and future graph layers.
repo_intelligence/state/	Ephemeral indexing state, change detection, and cache bookkeeping.
repo_intelligence/tests/	Unit, integration, fixture, and regression tests.
fixtures/repos/	Small representative repositories used in tests and validation.

6. Data Contracts

The data contracts below are illustrative. They describe structure and required fields without forcing a specific implementation language or serialization format.

Contract	Required fields	Notes
RepoSnapshot	repository_id, root_path, snapshot_id, created_at, file_count, folder_count, content_hash	Represents a complete indexed state.
FileRecord	path, normalized_path, file_type, language, size_bytes, hash, summary_ref, dependency_refs	One record per tracked file.
FolderRecord	path, child_file_count, child_folder_count, dominant_languages, summary_ref	Used for fast navigation.
DependencyHint	source_path, target_path, hint_type, confidence, evidence	Shallow signal only; not a full graph edge.

Contract	Required fields	Notes
ChangeEvent	change_type, path, previous_hash, current_hash, timestamp	Feeds incremental indexing.
QueryRequest	query_type, scope, path_prefix, filters, freshness	Used by other agents to request repository facts.
QueryResponse	matched_records, summaries, pagination, snapshot_id	Should be compact and deterministic.

Example contract sketch

Structured example

```
FileRecord:
  path: src/auth/login.py
  normalized_path: src/auth/login.py
  file_type: source
  language: python
  hash: 9a1f...c20
  summary_ref: summary:file:src/auth/login.py
  dependency_refs: [dep:src/auth/login.py->src/auth/token.py]
```

7. State Management

State should be split into ephemeral indexing state and persistent snapshot state. Ephemeral state tracks what is currently being scanned, cached, or invalidated. Persistent state stores the repository model that other agents will query.

The owner of the persistent state is the repository intelligence layer, not the calling agent. Callers should treat the snapshot as an immutable view and request a fresh snapshot only when they need newer facts.

State transitions must be driven by file hashes, git commit IDs, or explicit refresh requests so the same repository version always yields the same snapshot.

State type	Ownership	Persistence	Used by
Scan queue	Indexer runtime	Ephemeral	Scanner and classifier
File cache	Indexer runtime	Ephemeral with TTL	Change detection and summaries
Snapshot manifest	Repository Intelligence	Persistent	Planner, research, graph layers
Summary cache	Repository Intelligence	Persistent	Query service and downstream agents
Change journal	Repository Intelligence	Persistent	Incremental refresh logic

8. Component Specifications

Component	Purpose	Inputs	Outputs	Failure conditions	Future integration
RepoScanner	Discover all repository paths and apply ignore rules	Repository root, ignore config	Raw file and folder list	Must never mutate source files; must respect excludes; must be deterministic	Feeds PathNormalizer and ChangeDetector
PathNormalizer	Create canonical paths and stable IDs	Raw path, repo root	Normalized record keys	Should reject ambiguous paths	Used by every persisted record
FileClassifier	Identify file type, language, and role	Normalized path, extension, lightweight content probe	Classification labels	Must not overfit on heuristics; unknown stays unknown	Feeds summaries and future graph
DependencyHintExtractor	Collect shallow dependency hints	Text content, language classification	Import/include references	Should only emit hints with evidence	Consumes later in Part 10
SummaryGenerator	Write concise folder and file summaries	Classification, dependency hints, content snippets	Human-readable summary text and metadata	Must avoid long or speculative summaries	Feeds context compression
ManifestStore	Persist records and snapshots	Structured records and snapshot metadata	Stored manifest and retrieval index	Must preserve version integrity	Source of truth for queries
IncrementalIndexer	Refresh only changed records	Change events, previous snapshot	Updated snapshot and journal entries	Should not force full rebuild on every minor change	Critical for scale
RepositoryQueryService	Serve repository facts to agents	Query request, snapshot ID	Compact query response	Should time out cleanly on bad queries	Primary consumer interface

9. Implementation Sequence

Step 1: define contracts and snapshot schema

The rest of the layer depends on stable records. Build the data model first so later code writes to one shape.

Expected output: repository, file, folder, dependency, summary, and change event contracts.

Depends on: nothing except the architecture assumptions.

Step 2: implement repository scanning and ignore handling

You cannot classify or summarize what you cannot reliably enumerate.

Expected output: deterministic file and folder discovery for a known repository root.

Depends on: path normalization rules.

Step 3: implement file classification

Downstream features need a fast way to tell source files from docs, build files, tests, and generated artifacts.

Expected output: stable labels for file type and language.

Depends on: scanning and path normalization.

Step 4: implement shallow dependency hints

Agents need quick signals about module relationships before the graph layer exists.

Expected output: import/include references with evidence fields.

Depends on: classification and lightweight content reads.

Step 5: implement summary generation

Summaries make the repository model usable by planning and research layers without reopening large files.

Expected output: concise file and folder summaries tied to stable IDs.

Depends on: classification and hints.

Step 6: implement manifest persistence

The repository model must survive process restarts and support repeatable reads.

Expected output: snapshot storage, retrieval, and version metadata.

Depends on: record contracts.

Step 7: implement incremental refresh

A full rescan on every change will not scale and will make the layer unusable on larger repos.

Expected output: change journal, refresh detection, and partial update flow.

Depends on: persistent manifest and hashes.

Step 8: implement query service

Other agents should ask for repository facts using narrow queries instead of reading the whole snapshot.

Expected output: read-only API for prefix, type, language, and summary queries.

Depends on: manifest store and snapshot IDs.

Step 9: add fixture-driven integration tests

The repository layer is only useful if it behaves the same on every run.

Expected output: repeatable validation across small representative repos.

Depends on: full module set.

10. Validation Strategy

Validation should happen at three levels: unit tests for individual helpers, integration tests for repository scans, and regression tests against fixture repositories.

A strong validation set should include tiny repositories, mixed-language repositories, repositories with generated files, and repositories with intentionally messy directory layouts.

Test level	What it verifies	Example checks
Unit	Path normalization, classification, summary formatting	Stable IDs, correct language labels, summary length limits
Integration	End-to-end scan and persistence	All tracked files present in manifest, snapshot IDs stable
Regression	Behavior across known repositories	Ignored paths remain ignored, no duplicate records

Test level	What it verifies	Example checks
Performance	Scale and incremental update behavior	Refresh only changed files, query response remains bounded

11. Deliverables

- ☐ Repository scanner module
- ☐ Path normalization module
- ☐ File and folder classification module
- ☐ Shallow dependency hint extractor
- ☐ Summary generation module
- ☐ Persistent manifest store
- ☐ Incremental indexing and change journal
- ☐ Repository query API
- ☐ Representative repository fixtures
- ☐ Automated test suite for repository intelligence
- ☐ Developer documentation for the repository snapshot contract

12. Acceptance Criteria

- ☐ A repository scan produces a manifest containing every included file and folder exactly once.
- ☐ Ignored files and directories are excluded consistently across repeated scans.
- ☐ Each source file receives a stable normalized path, file type, and language label.
- ☐ At least one summary exists for every included source file and every non-trivial folder.
- ☐ Dependency hints contain evidence and confidence fields and do not pretend to be a full graph.
- ☐ A change to a single file updates only the affected records in the next incremental refresh.
- ☐ The query service can return file, folder, and path-prefix results without rescanning the repository.
- ☐ A restart of the service can reload the most recent snapshot and serve the same repository view.
- ☐ Regression tests pass on all supplied fixtures without path drift or duplicate entries.

Measurable completion rule

The part is complete only when a fresh scan, an incremental refresh, and a query round-trip all succeed on representative fixture repositories and the stored snapshot matches the expected contract.

13. Common Failure Modes

- Treating every dependency hint as a real dependency edge. That turns noisy heuristics into fake certainty.
- Producing summaries that are too long. If the summary is basically a reprint of the file, it is not a summary.
- Re-scanning the entire repository for tiny changes. That works on toy repos and fails on anything interesting.
- Using unstable IDs based on execution order rather than stable normalized paths and content hashes.
- Mixing repository intelligence with graph logic too early. This part should stay shallow and consistent.
- Ignoring generated files and vendor directories, which creates noise and hurts query quality.
- Storing ephemeral scan state as if it were durable truth. It will break restarts and confuse downstream consumers.

14. Future Integration

Part 10, the Code Knowledge Graph, should reuse the repository model as its input map rather than discovering the repository again from scratch.

Part 11, Context Compression, should consume folder summaries and file summaries when it decides what context to keep and what to omit.

Future planning and research agents should use the query API to ask narrow questions before they open files, which keeps the context window focused.

Later benchmark and evaluation layers should measure how well repository intelligence improves file selection accuracy and reduces irrelevant reads.

Downstream consumers

Repository Intelligence

- > Code Knowledge Graph
- > Context Compression
- > Planner / Research Agents
- > Evaluation and Benchmarking

15. Completion Checklist

- ☐ Repository scan produces a complete and deterministic manifest.
- ☐ File and folder summaries are generated and persisted.
- ☐ Dependency hints are stored with evidence and confidence.
- ☐ Incremental refresh updates only changed content.
- ☐ Query API returns compact, structured responses.
- ☐ Persistent snapshot reloads cleanly after restart.
- ☐ Regression fixtures pass without duplicate records or path drift.
- ☐ Part 10 can consume repository output without re-discovering the tree.

Final note

The Repository Intelligence layer is successful when it makes the rest of the system less stupid. That is the whole point: fewer blind scans, fewer guesses, and a much better starting point for deeper code understanding.